SCALAR Appendix

August 18, 2025

1 Introduction

This appendix provides a comprehensive overview of SCALAR, a web-based tool for semantic clustering and analysis of requirements engineering data, along with the extensive experimental framework that informed its design decisions. The document details the rigorous evaluation of eleven embedding models, ranging from traditional approaches like TF-IDF and Word2Vec to advanced transformer-based architectures such as Sentence-BERT and Sentence-RoBERTa, which were assessed using information retrieval metrics including Mean Reciprocal Rank (MRR) and Normalized Discounted Cumulative Gain (NDCG) to determine optimal text representations for CrowdRE data. The experimental analysis encompasses systematic performance comparisons across four clustering algorithms (K-Means, Gaussian Mixture Models, BIRCH, and Hierarchical Agglomerative Clustering) evaluated on binary through quinary classification tasks, revealing that transformer-based embeddings paired with probabilistic clustering methods achieve superior semantic coherence in requirements classification. Additionally, the appendix describes the development of domain-specific corpora through automated Wikipedia extraction using carefully curated noun phrases across five domains (Health, Safety, Energy, Entertainment, and Other) and a user-input domain-specific corpus, which serves as a semantic anchor for enhanced cluster labeling. The technical implementation details of SCALAR's Flask-based architecture, encompassing both frontend and backend functionalities, demonstrate how these research findings translate into a practical tool that enables users to upload .csv files, configure analysis parameters, and obtain interpretable clustering results through interactive visualizations and exportable formats.

2 Selection of Sentence Embedding

We evaluated the eleven embedding models (Table 1) , including TF-IDF, Word2Vec, and transformer-based architectures such as Sentence-BERT and Sentence-RoBERTa, to identify optimal representations on the SmartHome Crowd Requirements dataset(1).

Table 1: Embedding Model Details

Model Name	**Model Version / ID**	**Dimensions**
TF-IDF (1,2-gram)	N/A	N/A
Word2Vec	Google News (300 dimensions)	300
FastText	Wikipedia (300 dimensions)	300
GloVe	Gigaword (300 dimensions)	300
Sentence-BERT	paraphrase-MiniLM-L6-v2	384
Sentence-RoBERTa	all-distilroberta-v1	768
LaBSE	sentence-transformers/LaBSE	768
USE	Universal Sentence Encoder (v4)	512

The selection of optimal sentence embeddings was guided by rigorous evaluation using two key information retrieval metrics: Mean Reciprocal Rank (MRR) and Normalized Discounted Cumulative Gain (NDCG). These metrics quantitatively assess how effectively each embedding model retrieves semantically relevant requirements when given a query requirement.

For MRR, each requirement in the dataset serves as a query, and all other requirements are ranked based on their cosine similarity to the query embedding. The reciprocal rank of the first correctly matching requirement (sharing the same domain label) is recorded. The MRR score represents the average of these reciprocal ranks across all queries, with values ranging from 0 to 1. A perfect MRR of 1 indicates that for every query, the most semantically similar matching requirement appears first in the ranked list.

$$MRR = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{FirstRank_i} \tag{1}$$

The NDCG metric provides a more nuanced evaluation by considering the graded relevance of all retrieved requirements, not just the first correct match. For each query, the Discounted Cumulative Gain (DCG) is computed by summing the relevance scores (1 for matching domain, 0 otherwise) of retrieved requirements, with logarithmic discounting applied to lower ranks. This is normalized by the Ideal DCG (IDCG), which represents the maximum possible DCG achievable with perfect ranking. The final NDCG score is the average of these normalized values across all queries.

$$NDCG = \frac{1}{N} \sum_{i=1}^{N} \frac{DCG_i}{IDCG_i} \tag{2}$$

3 Experimental Results

3.1 Embedding Selection

The selection of optimal embeddings was guided by a rigorous evaluation using information retrieval (IR) metrics Mean Reciprocal Rank (MRR) and Normalized Discounted Cumulative Gain (NDCG) as detailed. Transformer-based models, particularly Sentence-RoBERTa, emerged as top performers, achieving MRR, NDCG, and F1 scores of 0.737, 0.771 and 0.680, respectively, thus significantly outperforming traditional embeddings like TF-IDF and Word2Vec. These models excelled in capturing contextual relationships, as

reflected in their scores (Table 3), which measure ranking quality by accounting for graded relevance. Based on these findings, we focused our expanded clustering experiments on Sentence-BERT and Sentence-RoBERTa as our primary embedding models for evaluation across all four clustering algorithms.

Embedding	**MRR**	**NDCG**	**F1-Score**
TF-IDF (1,2-gram)	0.695	0.769	0.616
Word2Vec (Pre-trained)	0.715	0.770	0.652
TF-IDF Word2Vec (Pre-trained)	0.714	0.770	0.648
FastText (Pre-trained)	0.713	0.770	0.635
TF-IDF FastText (Pre-trained)	0.706	0.770	0.624
GloVe (Pre-trained)	0.710	0.770	0.639
TF-IDF GloVe (Pre-trained)	0.702	0.770	0.629
Sentence-BERT	0.729	0.771	0.666
Sentence-RoBERTa	0.737	0.771	0.680
LaBSE	0.727	0.771	0.667
USE	0.723	0.772	0.666

Table 2: Performance comparison of different embeddings.

3.2 Domain-Specific Corpus Extraction

To enrich clustering with domain context, Wikipedia articles were systematically retrieved using a breadth-first search (BFS) strategy, seeded with domain-specific noun phrases derived from an analysis of the first 100 datapoints per class label. The following terms guided corpus extraction for each experimental domain:

- **Health**: Smart fitness trackers, Automated medication reminders, Health monitoring systems, Medical alert devices, Smart scales, Sleep trackers, Heart rate monitors, Blood pressure monitors, Exercise equipment with sensors, Nutrition tracking apps.
- **Safety**: Smart security cameras, Automated door locks, Home alarm systems, Motion detectors, Fire alarms, Carbon monoxide detectors, Emergency response systems, Child safety devices, Surveillance equipment, Facial recognition systems.
- **Energy**: Smart thermostats, Automated lighting systems, Energy monitoring devices, Solar panels, Smart power outlets, Automated blinds, Energy-efficient appliances, Smart grid systems, Power management solutions, Automated climate control.
- **Entertainment**: Voice-controlled music systems, Smart TVs, Gaming consoles, Streaming media players, Virtual reality headsets, Home theater systems, Interactive displays, Digital assistants for entertainment, Smart speakers, Audio systems.
- **Other**: Smart grocery lists, Automated pet feeders, Smart gardening systems, Home organization tools, Smart kitchen appliances, Cleaning robots, Smart mirrors, Smart furniture, Communication systems, Smart laundry systems.

Starting from these seed queries, the algorithm explored linked Wikipedia pages up to a depth of 4, excluding disambiguation pages. A limit of 25 pages per domain was enforced

to maintain relevance. Extracted content underwent post-processing to filter short or off-topic sentences, retaining only those containing domain keywords (e.g., "monitor" in health contexts). The domain-specific corpora are stored temporarily within the session-data structure when the pipeline is executed. It is not explicitly stored on disk as a separate file. This ensured the corpus served as high-quality semantic anchors for cluster labeling while mitigating noise from generic Wikipedia content.

3.3 Model Performances

This section validates the efficacy and generalizability of SCALAR through four crowd-based requirements datasets from different domains. The first dataset, called the Garmin Connect dataset (2), comes from user reviews of the Garmin Connect application on the Google Play Store. The second dataset, called the Global Platform Specification (GPS) dataset(3), comes from the GlobalPlatform organization. The third dataset, called the FinalAnnotatedReviews dataset(4), contains labeled reviews from user feedback of 108 AI-based apps from the Google Play Store. The fourth dataset is the SmartHome Crowd Requirements dataset(1), which was collected as part of research to understand the influence of personality and creativity potential on CrowdRE. Additional details on the datasets are given in Table 3.

Table 3: Dataset Overview with Requirements and Labels

Dataset Name	**Total Require-ments**	**Labels**	**Count**
Garmin Connect	768	Problem Reports	586
		Feature Requests	182
Global Platform Specifications	175	Security Features	62
		Non-Security Features	113
Final Annotated Reviews	2605	Fair Reports	1132
		Non-Fair Reports	1473
SmartHome Crowd Requirements	2966	Safety	892
		Energy	626
		Health	593
		Entertainment	471
		Other	384

The comprehensive evaluation across four clustering algorithms K-Means, GMM, BIRCH, and HAC on the SmartHome Crowd Requirements dataset using Wikipedia-extracted domain-specific corpus for dynamic labeling revealed distinct performance patterns that guided subsequent algorithm selection. For binary classification tasks, K-Means and GMM consistently outperformed BIRCH and HAC, with GMM achieving the highest scores in several cases (e.g., 0.9104 for energy vs. entertainment with Sentence-RoBERTa) as shown in Table 4. This superiority stems from GMM's (5) probabilistic

Table 4: Comprehensive Comparison of F1 Scores Across Different Clustering Algorithms and Embedding Models on the SmartHome Crowd Requirement Dataset

Clustering Type	Cluster Labels	KMeans		GMM		BIRCH		HAC	
		SBERT	SRoBERTa	SBERT	SRoBERTa	SBERT	SRoBERTa	SBERT	SRoBERTa
Binary	energy vs entertainment	**0.9110**	**0.9072**	**0.9073**	**0.9104**	0.8299	0.8415	**0.8971**	**0.8726**
	energy vs safety	0.8813	0.8825	0.8760	0.9015	0.7760	**0.8834**	0.8033	0.3940
	entertainment vs safety	0.8952	0.8965	0.8838	0.8753	**0.8802**	0.8556	0.7611	0.8644
	health vs energy	0.7881	0.7879	0.7866	0.7883	0.7584	0.7402	0.7813	0.7938
	health vs entertainment	0.8625	0.8653	0.8578	0.8603	0.7584	0.8562	0.8626	0.8067
	health vs safety	0.8025	0.7952	0.8114	0.8133	0.7137	0.7986	0.7154	0.3158
	energy vs other	0.7996	0.7898	0.8051	0.8011	0.7124	0.7495	0.7433	0.7573
	entertainment vs other	0.7581	0.7639	0.7788	0.7759	0.7750	0.7408	0.6886	0.7243
	safety vs other	0.7953	0.7560	0.7715	0.6602	0.7623	0.7199	0.7653	0.7057
	health vs other	0.5993	0.6079	0.6140	0.6194	0.7109	0.7123	0.6398	0.7114
Tertiary	energy vs entertainment vs safety	**0.8064**	**0.8074**	**0.8025**	**0.8034**	**0.7113**	**0.7355**	**0.7138**	0.5234
	health vs entertainment vs safety	0.5254	0.5402	0.7621	0.7495	0.4384	0.4541	0.5273	0.5414
	health vs energy vs entertainment	0.7404	0.7401	0.7416	0.7406	0.6535	0.7228	0.4356	0.6431
	health vs energy vs safety	0.7487	0.7425	0.7527	0.7444	0.6619	0.6866	0.6812	**0.7084**
	energy vs entertainment vs other	0.7294	0.7293	0.7287	0.7281	0.6791	0.6811	0.6101	0.6541
	energy vs safety vs other	0.7204	0.7250	0.7296	0.7281	0.6107	0.6798	0.6553	0.6938
	entertainment vs safety vs other	0.4809	0.5346	0.4820	0.6837	0.4142	0.5177	0.5242	0.5698
	health vs energy vs other	0.5548	0.5715	0.4092	0.5700	0.5396	0.5477	0.5530	0.5464
	health vs entertainment vs other	0.5003	0.5409	0.5239	0.5505	0.3998	0.4973	0.4759	0.5201
	health vs safety vs other	0.5401	0.5410	0.5951	0.5392	0.4670	0.5198	0.5821	0.5718
Quaternary	health vs energy vs entertainment vs safety	**0.7109**	**0.7048**	**0.7108**	**0.7002**	0.5033	0.5120	**0.5513**	0.3670
	energy vs entertainment vs safety vs other	0.7020	0.6904	0.6068	0.5682	**0.5641**	0.5123	0.5230	0.5241
	health vs energy vs entertainment vs other	0.5325	0.5608	0.4692	0.5445	0.4720	0.5015	0.4823	**0.5344**
	health vs energy vs safety vs other	0.4890	0.5432	0.5964	0.5970	0.4778	**0.5548**	0.5428	0.3478
	health vs entertainment vs safety vs other	0.4772	0.5436	0.5831	0.5649	0.4106	0.3742	0.3537	0.3564
Quinary	health vs energy vs entertainment vs safety vs other	0.5085	0.5500	0.4978	0.4977	0.4860	0.4738	0.4693	0.4131

approach, which better handles the fuzzy semantic boundaries inherent in natural language requirements. K-Means followed closely behind, with scores like 0.9110 for energy vs. entertainment using Sentence-BERT, demonstrating its effectiveness despite its simpler mathematical foundation. In tertiary classification scenarios, GMM maintained strong performance, particularly in complex domain combinations such as health vs. safety vs. entertainment (0.7621 with Sentence-BERT), while BIRCH and HAC consistently underperformed across most classification scenarios, particularly in quaternary and quinary classifications where its hierarchical merging strategy proved less effective for high-dimensional transformer embeddings. As classification complexity increased from binary to quinary, all algorithms showed performance degradation, though GMM and K-Means maintained relatively higher F1-scores, with both achieving approximately 0.50 in the most complex quinary classification using Sentence-RoBERTa, significantly outperforming BIRCH (0.4738) and HAC (0.4131). This evaluation identified K-Means and GMM as the two superior algorithms for subsequent testing on domain-specific datasets.

The evaluation on the remaining three domain-specific datasets using stratified 70-30 splits, where the smaller 30% portion served as domain-specific corpus for dynamic labeling, demonstrated improved performance over Wikipedia-extracted corpus due to higher domain similarity scores. On the Garmin Connect dataset for feature request vs. problem report classification, K-Means achieved superior performance with F1-accuracy scores of 0.76 (Sentence-BERT) and 0.78 (Sentence-RoBERTa), outperforming GMM's scores of 0.68 and 0.79 respectively, as detailed in Table 5 and Table 6. For the Global Platform Specification dataset focusing on security vs. non-security requirements, both algorithms showed comparable performance, with K-Means achieving 0.70 (Sentence-BERT) and 0.75 (Sentence-RoBERTa), while GMM recorded 0.69 and 0.78 respectively. The Final Annotated Reviews dataset demonstrated consistent performance across both algorithms, with identical F1-accuracy scores of 0.70 for Sentence-BERT and variations for Sentence-RoBERTa (0.78 for K-Means, 0.70 for GMM). These results indicate that K-Means showed particular robustness across diverse application contexts when using stratified domain-specific corpus extraction, while the domain-specific approach consistently yielded higher similarity scores and better clustering performance compared to

generic Wikipedia-based corpus extraction, validating the importance of domain-tailored dynamic labeling in unsupervised CrowdRE classification.

Table 5: KMeans Clustering Performance Across Datasets

Dataset	**Cluster Type**	**Cluster Label**	**KMeans (F1)**	
			SBERT	**SRoBERTa**
Garmin Connect	Binary	Feature Request vs Problem Report	0.76	0.78
Global Platform Spec.	Binary	Security vs Non-Security Req.	0.70	0.75
Final Annotated Reviews	Binary	Fair vs Non Fair	0.70	0.70

Table 6: GMM Clustering Performance Across Datasets

Dataset	**Cluster Type**	**Cluster Label**	**GMM (F1)**	
			SBERT	**SRoBERTa**
Garmin Connect	Binary	Feature Request vs Problem Report	0.68	0.79
Global Platform Spec.	Binary	Security vs Non-Security Req.	0.69	0.78
Final Annotated Reviews	Binary	Fair vs Non Fair	0.69	0.70

4 Software description

4.1 Software architecture

In figure 1, we can observe the overall architectural framework of SCALAR[1], which is organized into three primary domains: *Upload*, *Parameters*, and *Results*. The system is built using Flask[2], a Python web framework, with data exchange facilitated through JSON.

4.1.1 Frontend Architecture

SCALAR's frontend is implemented using HTML5, CSS3, and ES6+ JavaScript without external frameworks, resulting in a lightweight single-page application. The frontend communicates with the backend through RESTful[3] endpoints that follow three distinct operational domains visualized in the system architecture:

[1] `https://github.com/SushovitNanda/SCLARA-Tool`

[2] `https://palletsprojects.com/projects/flask/`

[3] `https://www.geeksforgeeks.org/rest-api-introduction/`

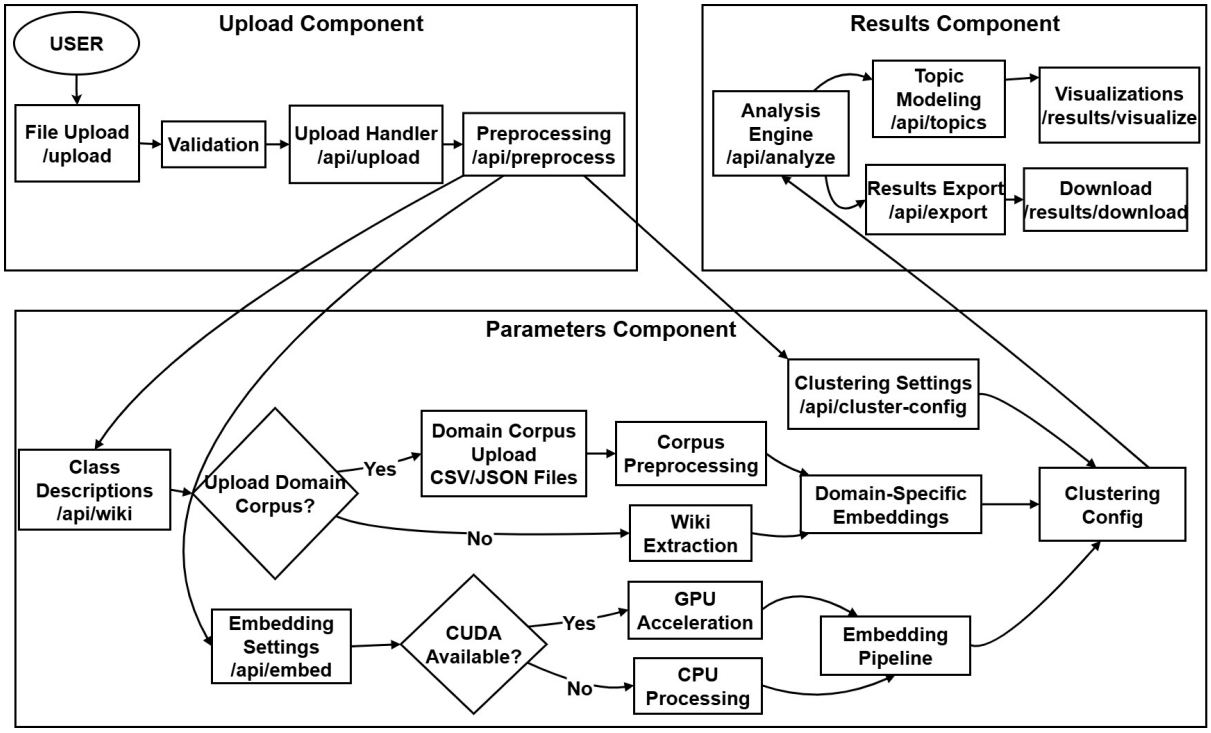


Figure 1: SCALAR Architecture

- Upload component: Manages document submissions (capped at 16MB files) and validation tasks. Preprocessing is handled using the Natural Language Toolkit (nltk)[4] library.
- Parameters component: Configures embeddings supporting conditional GPU acceleration contingent on CUDA[5] availability. Clustering parameters are managed with user input domain specific corpus or optional wiki-derived class descriptions.
- Results component: Executes analytical processes generates topic models, produces visualizations, and supports result exports with download capabilities.

The user interface offers intuitive, context-sensitive navigation, enabling seamless transitions between stages, revisiting parameter configurations, or initiating new analyses as required. The details of the implementation are provided in this appendix.

4.1.2 Backend Architecture

SCALAR's backend efficiently processes user requests, delivering real-time processing visibility via terminal output. The system is organized into three core operational components:

- Upload component: Handles initial data ingestion and preparation workflows. The system implements secure file management procedures, including type validation for CSV files, path traversal prevention through Werkzeug's *secure_filename*[6], and

[4] `https://www.nltk.org/`

[5] `https://developer.nvidia.com/cuda-toolkit`

[6] `https://tedboy.github.io/flask/generated/werkzeug.secure_filename.html`

temporary storage using Python's *tempfile* module. A 16MB file size limit is enforced at the server level via Flask configuration. Data persistence is managed through a hybrid session management system that combines in-memory storage with file-based backup. A global *session_data* dictionary maintains current application state, with periodic saves to *session_data.pkl* ensuring session recovery across server restarts. This approach optimizes speed through rapid memory access while guaranteeing reliability with persistent storage.

- Parameters component: Orchestrates the configuration and processing pipeline, seamlessly connecting data ingestion to result generation. It manages embedding generation with conditional GPU acceleration, dynamically routing tasks to GPU-accelerated pipelines when CUDA is available or to CPU-based alternatives when required. The embedding pipeline supports model selection, batch processing, and robust error handling, ensuring consistent output across diverse hardware environments. Clustering configuration is streamlined which validates and processes user-defined parameters for various clustering algorithms. Additionally, this component enables user to input their own domain specific corpus for each label or optional (0 or more pages) extraction of class descriptions from Wikipedia enriching analyses with external knowledge. Comprehensive parameter validation and error handling prevent invalid configurations from disrupting workflows. Security is prioritized with properly configured CORS[7] headers, rigorous input validation, session security via random secret key generation using *os.urandom(24)*, and secure HTTP headers, ensuring robust protection throughout the system.

- Results component: Executes core analytical processes, delivering actionable insights to users. The modular workflows, encompassing text preprocessing, embedding generation, clustering, and topic modeling. Asynchronous task execution, powered by Python's *threading* module, manages long-running analyses, with a dedicated task queue and progress tracking endpoints that keep users informed during complex operations. The results are cached for efficient retrieval, ensuring application responsiveness during intensive processing. The data export in multiple formats incorporates optimisations such as temporary file cleanup, resource and connection pooling, efficient serialisation, and caching. These enhancements enable the system to support concurrent users and complex analyzes while maintaining performance under heavy workloads.

4.2 Software functionalities

The *FrontEnd's* initial page features a user-friendly interface designed for uploading *.csv* files containing text data across multiple columns, which will later undergo semantic-based topic clustering and modeling. The interface is equipped with several key components, such as a file upload form that supports drag-and-drop functionality from the local machine for ease. Once the user submits a file, the *BackEnd*, managed by the */upload* endpoint, processes the file through a series of steps. First, it validates the file type to be a *.csv* file and checks that the size does not exceed the 16MB limit. The system then secures the filename using *secure_filename* to prevent potential security risks. After validation, the file is temporarily stored in a designated directory, and its file path is saved

[7] `https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS`

within the session data for further processing. Finally, the backend sends a response to be either a success confirmation or an error message back to the *FrontEnd*, ensuring the user is informed of the upload status.

After a successful file upload, the *FrontEnd* directs the user to the parameters page where they can configure various settings for the application process. The available options include selecting the embedding type (either Sentence-BERT or Sentence-RoBERTa) to generate semantic representations of the text data. Users can also choose a clustering method from several algorithms, such as K-means, HAC (Hierarchical Agglomerative Clustering), GMM (Gaussian Mixture Model), or BIRCH. Additionally, they must specify the number of clusters to generate, set a minimum wikipedia pages threshold for external validation (if applicable), and toggle BERTopic Analysis to enable or disable advanced topic modeling features. The interface also provides class descriptions for each cluster, helping users understand the expected output structure. When the user submits their selected parameters, the *BackEnd*, managed by the */set-parameters* endpoint, processes the request by first validating all inputs to ensure they meet the required range limits. Once validated, the parameters are stored in the session data for consistency across subsequent steps. The system then initializes the analysis process, preparing the data for embedding, clustering, and topic modeling. Finally, the *BackEnd* sends a success or error response back to the *FrontEnd*, informing the user whether the configuration was accepted or if adjustments are needed. This step ensures that the analysis proceeds only with valid and intended settings.

The *BackEnd* then initiates the text preprocessing pipeline as part of the analysis process (executed via *run_analysis_task* endpoint), which systematically refines the raw text data to improve the accuracy and efficiency of downstream topic modeling and clustering. The pipeline begins with text cleaning, where all characters are converted to lowercase to ensure uniformity, and special characters (such as punctuation and symbols) are removed to eliminate noise. Next, the text undergoes tokenization, splitting sentences into individual words or tokens. The tokens are then processed using part-of-speech (POS) tagging, which identifies grammatical categories (e.g., nouns, verbs) to help filter or prioritize meaningful terms. Following this, stopword removal eliminates common but insignificant words (e.g., "the," "and") that do not contribute to semantic meaning. The pipeline then applies lemmatization, reducing words to their base or dictionary form (e.g., "running" becomes "run") to consolidate variations of the same term. Finally, text normalization standardizes the processed text, ensuring consistency in formatting and structure before proceeding to embedding and clustering. This comprehensive preprocessing stage enhances the quality of the input data, leading to more coherent and interpretable topic modeling results.

The 'WikiCorpusExtractor' class of the *BackEnd* module, systematically gathers and refines relevant wikipedia content using the wikipedia python package to build a domain-specific knowledge corpus that enhances topic modeling. Initialized with class descriptions in parameters page (cluster labels and their semantic summaries), the extractor first identifies key terms through a multi-layered linguistic analysis. Using regex, it captures noun phrases (e.g., "machine learning"), proper nouns, plural forms, adverbs, and gerunds, while a Part-of-Speech (POS) tagger isolates nouns and filters stop words. To ensure domain relevance, the current cluster label (e.g., "Artificial Intelligence", "IoT", "Requirement Engineering") is injected into the search terms if absent. The extraction then proceeds with focused Wikipedia queries. For each term the system performs a breadth-first search (limited to 4 levels of depth) to fetch up to 5 related pages, avoiding

circular references via a *visited_pages* tracker. Disambiguation pages (e.g., "X may refer to:") are automatically discarded. Each page's raw content undergoes text preprocessing via the 'TextPreprocessor' module, and links from the page are recursively explored to expand the corpus while respecting rate limits (0.3–0.5s delays between requests to avoid api throttling). This corpus is refined through semantic filtering and sentences are retained only if they contain at least 10 words and match any key term from the class description, ensuring topical coherence. Progress is tracked via nested *tqdm* progress bars (one for domains, e.g., clusters; and another for individual terms), providing real-time feedback. The final output is a structured dictionary where each cluster label maps to a consolidated text corpus of wikipedia-derived knowledge, which serves as a contextual foundation for downstream tasks like cluster labeling or semantic validation. Error handling (e.g., invalid pages, network issues) is robust, with logged exceptions preventing pipeline disruption.

The embedding generation process begins by dynamically selecting the appropriate computational device, prioritizing GPU acceleration with CUDA if available, otherwise defaulting to CPU. The 'EmbeddingGenerator' class is initialized with a specified embedding type (either "sentence-bert" or "sentence-roberta") after robust analysis between 9 embedding models via various information retrieval metrics such as F1 score, MRR and NDCG. A predefined model mapping provides multiple fallback options for each embedding type. For instance, "sentence-bert" defaults to "sentence-transformers/all-MiniLM-L6-v2" but includes alternatives like "distilbert-base-nli-stsb-mean-tokens" to handle potential load failures. If the requested embedding type is unrecognized, the system defaults to "sentence-bert" with a logged warning. Once the model is loaded, the *generate_embeddings* method processes input texts in batches (default size 32) for efficiency, first cleaning each text via the 'TextPreprocessor' to standardize formatting. Invalid or empty texts are replaced with "unknown" to maintain consistency. The method handles batch processing with error resilience. If encoding fails for a batch, it logs the error and substitutes zero embeddings for that batch, preventing total failure. Progress is logged for each batch, providing transparency during the operation. The final output is a stacked array of embeddings, with dimensionality matching the model's specifications (e.g., 384 dimensions for MiniLM). If errors occur, the method returns zero embeddings for all inputs, ensuring that downstream processes receive valid data structures. The 'DummyEncoder' serves as a contingency, mimicking the interface of a true transformer model while generating placeholder embeddings, thus maintaining system stability even during complete model load failures.

The clustering process begins by validating input embeddings and adjusting parameters to handle edge cases. If the number of samples is insufficient for the requested clusters, it automatically reduces the cluster count while ensuring at least two clusters remain, and if only one sample exists, it bypasses clustering entirely and assigns a default label. The embeddings are standardized using 'StandardScaler' to ensure consistent scaling across different clustering methods, which include K-means (fixed random state and multiple initializations for stability), HAC (calculates centroids post-clustering), GMM (probabilistic clustering with predefined components), and BIRCH (hierarchical clustering with a fixed threshold). Each method logs cluster distribution for transparency, and if fewer clusters are formed than requested, the system intelligently splits the largest clusters by reassigning half their points to new synthetic clusters, ensuring the target cluster count is met while maintaining meaningful groupings. After clustering, the system constructs a similarity matrix by computing cosine similarities between cluster centroids and

pre-generated wikipedia label embeddings, filling missing label embeddings with zeros if necessary. The enhanced greedy allocation algorithm then assigns labels to clusters, incorporating optional z-score normalization (for outlier-resistant scaling) and softmax transformation (to convert similarities into probabilistic distributions) for more robust matching. This algorithm prioritizes high-similarity pairs while preventing duplicate label assignments, ensuring each cluster receives the most semantically relevant label. Finally, the predicted labels are mapped back to the original samples, with detailed logging of cluster-label assignments for debugging.

The BERTopic analysis process, when enabled, performs hierarchical topic modeling on each cluster independently, adapting its parameters to the specific characteristics of each cluster's documents. The system begins by initializing the 'BERTopicInterpreter' module with configurable parameters including an embedding model (defaulting 'all-MiniLM-L6-v2'), n-gram range (default 1-3), and a base minimum topic size (automatically adjusted per cluster). For each cluster, the system first filters and validates documents, removing empty or overly short texts, and skips clusters with insufficient valid data, falling back to basic keyword extraction if fewer than 5 documents are available. For clusters with adequate data, the system dynamically configures a cluster-specific BERTopic model, optimizing parameters like *min_topic_size* (scaled to cluster size), UMAP dimensionality ('n_components'), and nearest neighbors ('n_neighbors'). The model processes documents to identify topics by leveraging ClassTF-IDF for term weighting and MMR to diversify topic representations (if available). Topics are extracted and filtered to remove numeric-only terms and outliers, retaining the top 10 most representative words per topic. If modeling fails, the system resorts to countvectorizer-based keyword extraction as an alternative, ensuring every cluster produces interpretable output.

The visualization generation system of the *BackEnd* produces multiple plots to analyze and interpret clustering results. The 2D cluster visualization (from 'visualize_clusters') projects high-dimensional embeddings into two principal components using PCA, comparing cluster assignments (either string labels or numeric IDs) against spatial distribution in reduced dimensions. The x and y axes represent the first and second principal components, with their explained variance ratios (shown in axis labels) indicating how much original variance each component captures. The higher values (closer to 1.0) suggest better preservation of the original structure. Cluster centroids, if available, are marked with black "x" symbols, and points are colored by cluster membership, allowing visual assessment of separation and overlap between groups. The similarity matrix plot (from 'plot_similarity_matrix') compares clusters (rows) against predefined class labels (columns) using a heatmap of cosine similarity scores (ranging from 0 to 1), where higher values (darker orange) indicate stronger semantic alignment between a cluster centroid and a class's wikipedia-derived embedding. Annotated scores quantify these relationships, suggesting meaningful matches. The cluster size distribution plot (from 'plot_cluster_sizes') compares cluster IDs (x-axis) against sample counts (y-axis) in a bar chart, revealing imbalances in group sizes. Annotated bar heights show exact counts, aiding in identifying underpopulated or dominant clusters. Finally, the topic hierarchy visualization (from 'visualize_hierarchy') compares terms (y-axis) against their importance scores (x-axis) within a cluster's topic, using a horizontal bar chart. Scores are derived from BERTopic's c-TF-IDF or fallback term frequencies, normalized to reflect relative scoring. This plot highlights key terms defining each topic, ordered by descending weight to emphasize dominant themes. Together, all these visualizations enable comprehensive

evaluation of clustering quality, semantic coherence, and topic interpretability.

Finally, the */results* page of the *FrontEnd* systematically presents all analytical outputs. Upon completion of clustering and topic modeling, the system aggregates cluster assignments, topic summaries, visualizations, distribution statistics, and similarity matrix. These results are stored in session-specific data structures with checksums and are exposed via RESTful endpoints. The */results* page returns a condensed overview for *FrontEnd*, while */download-results* and */download-json* provide *.csv* and *.JSON* exports respectively, the latter including raw embeddings and model parameters for reproducibility. The *FrontEnd* dynamically renders this data through interactive components.

Throughout application execution, the background threads manage resource-intensive tasks (embedding generation, BERTopic modeling) with periodic status updates (e.g., notifications for progress percentages). The system employs continuous operations for session data updates, ensuring consistency even during mid-process failures. Error handling follows a robust approach. Debug logs capture full stack traces, model configurations, and intermediate data dumps (e.g., preprocessed text samples), stored separately from user-visible logs to balance transparency and security. The results are cached in *Redis* for active sessions and archived to cloud storage (S3-compatible) for long-term access, with automatic cleanup policies for stale data.

This complete pipeline transforms raw text data into meaningful clusters and topics, providing users with both visual and analytical insights into their data's semantic structure. The end-to-end design ensures reliability from analysis to visualization, maintaining usability under load while providing detailed diagnostics via terminal for maintenance.

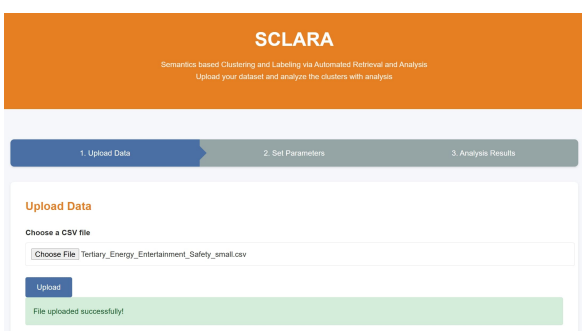
5 Illustrative examples

Instructions for executing the tool are detailed in the GitHub README.md[8] file. Users begin at the upload interface (Figure 2a), where they can submit CSV files, such as the 571KB CrowdRE dataset, via a user-friendly drag-and-drop or file selection mechanism. This intuitive interface serves as the entry point for dataset submission.

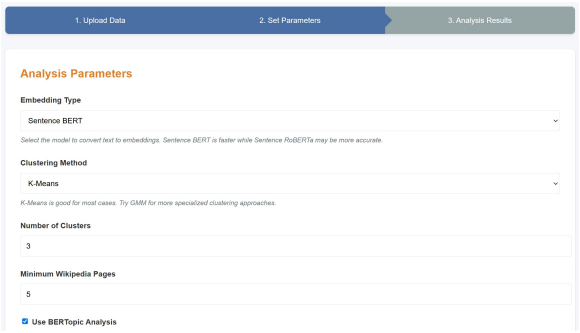
After uploading, users configure analysis settings through the parameter interface (Figure 2b). This panel allows selection of an embedding model—Sentence-BERT for faster processing or Sentence-RoBERTa for greater accuracy—and a clustering algorithm (K-Means or Gaussian Mixture Models) based on analytical requirements. Users can specify the number of clusters (default: 2), set a minimum Wikipedia pages threshold for validation (default: 3), and enable optional BERTopic analysis for advanced topic modeling. For domain-specific corpus extraction and robust cluster labeling, users must update the class description, as shown in Figure 2c. Analysis is initiated directly from this interface. During processing, a loading screen (Figure 3a) provides a lazy loading mechanism to indicate ongoing activity, while computational and processing details are displayed in the terminal.

Upon completion, the tool presents results through integrated visualizations and data tables. A cluster distribution table (Figure 3b and Figure 3e) provides a quantitative breakdown of data segmentation. A Principal Component Analysis (PCA) visualization (Figure 3c) displays clusters as distinct colored groups in two-dimensional space, with explained variance ratios to assess cluster separation quality. The results interface includes a similarity matrix (Figure 3d) that quantifies the alignment between automated

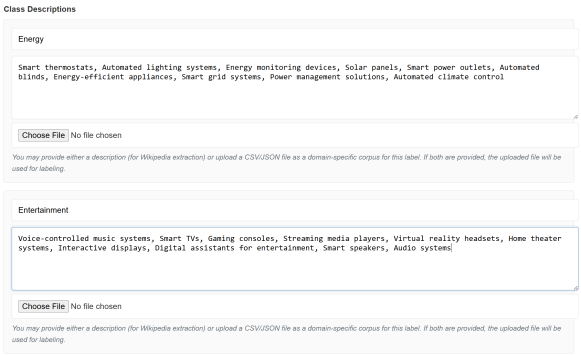
[8] `https://github.com/SushovitNanda/SCLARA-Tool/blob/main/README.md`



(a) Data upload interface



(b) Parameter configuration

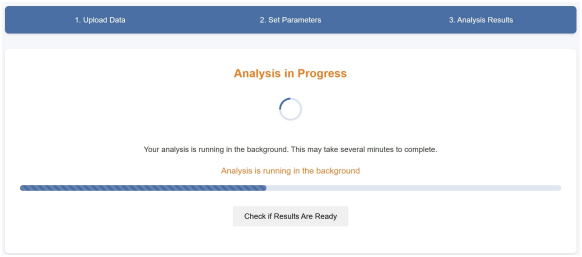


(c) Class description analysis

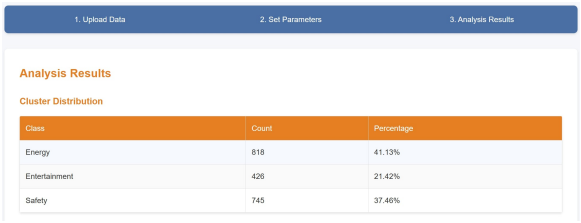
Figure 2: SCLARA tool interface screens showing the workflow from data upload to parameters analysis

clustering results and expected categories, enabling users to assess analysis accuracy. The topic analysis section (Figure 3f) highlights the most representative terms for each cluster, helping users identify the semantic themes defining each group.

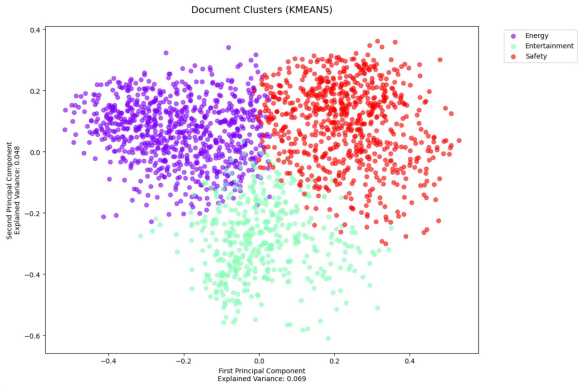
At the bottom of the results interface, users can export clustering results in CSV format for spreadsheet applications or JSON format for programmatic use, as shown in Figure 3f. The JSON export includes clustering results, raw embeddings, and model parameters, ensuring full reproducibility and seamless integration with external analysis workflows.



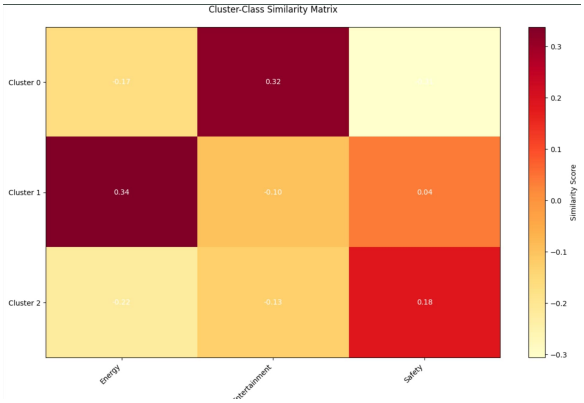
(a) Loading screen



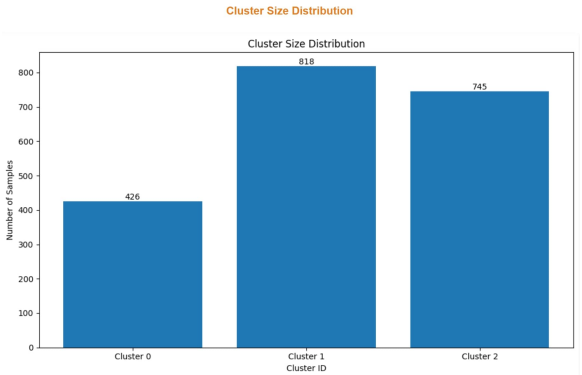
(b) Cluster distribution



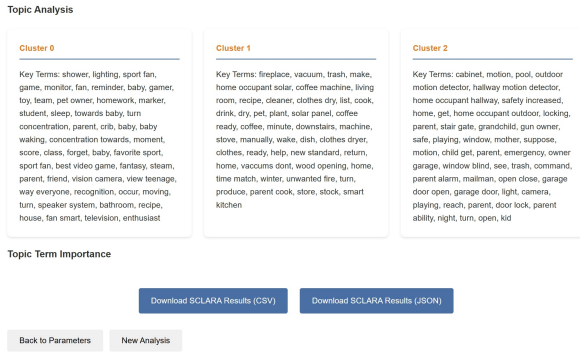
(c) Cluster diagram



(d) Similarity matrix



(e) Cluster size distribution



(f) Topic analysis results

Figure 3: SCLARA tool workflow and results visualization, showing the process from data loading to cluster analysis and topic modeling.

6 Limitations

The tool's primary limitations stem from its Wikipedia dependency, which restricts content to English sources and introduces linguistic ambiguities, categorization constraints, and ±2-3% metric variations that cannot be resolved without an architectural overhaul. Wikipedia's open-source nature allows non-expert, potentially biased users to edit content and arbitrarily add or remove pages, compromising data quality and reliability for serious analytical purposes. Single-threaded processing and API rate limits create scalability bottlenecks. User expertise requirements significantly impact effectiveness through inaccurate class descriptions, domain knowledge gaps, and parameter misunderstanding that mitigation strategies only partially address. Technical constraints include limited clustering algorithms, restricted embedding options, and measurement inconsistencies from Wikipedia's categorization system, with generalizability severely restricted by the

Wikipedia-centric approach and single-machine architecture.

7 Threats to validity

Internal validity threats arise from technical constraints and user limitations affecting measurement reliability. Wikipedia dependency, API limits, single-threaded processing, and ±2-3% metric variation remain unmitigated due to external dependencies and architectural constraints. User expertise issues including inaccurate class descriptions, domain knowledge gaps, and parameter understanding are partially mitigated through BSF search algorithms, query templates, validation checks, and automated optimization. Technical constraints receive mixed mitigation: file size limits and CSV-only input are fully resolved through chunking and format conversion, while clustering algorithms and embedding options are partially improved through parameter tuning. User challenges with time constraints and technical limitations are addressed via simplified interfaces, guided workflows, and automated analysis pipelines.

External validity limitations stem from generalizability constraints and interpretation challenges. Wikipedia's English focus, linguistic ambiguities, and single-machine processing remain unmitigated without major architectural changes. User difficulties interpreting results, selecting metrics, and understanding terminology are partially mitigated through simplified visualizations, automated metric selection, glossaries, and contextual help. Web interface restrictions are mixed: accessibility issues are fully resolved through responsive design while visualization capabilities are partially enhanced. User challenges generalizing across domains and applying results practically are addressed through domain-specific templates, cross-validation, use-case examples, and application guides.

Construct validity challenges involve measurement accuracy and user interpretation issues. Wikipedia's categorization limitations, temporal dynamics, and library dependencies remain unmitigated due to platform constraints. User subjectivity in clustering quality assessment, feature selection bias, and validation difficulties are partially mitigated through objective metrics, benchmark comparisons, feature importance analysis, and domain-specific validation rules. Measurement standardization shows mixed results: basic validation mechanisms are fully implemented through cross-validation and comparative analysis, while quality metrics are partially improved through multiple evaluation criteria. User challenges establishing causal relationships and maintaining consistency are addressed through correlation analysis, causal inference tools, standardized procedures, and analysis templates, though preprocessing pipelines remain partially constrained despite optimization implementations.

References

[1] P. K. Murukannaiah, N. Ajmeri, and M. P. Singh, "Acquiring creative requirements from the crowd: Understanding the influences of personality and creative potential in crowd re," in *2016 IEEE 24th International Requirements Engineering Conference (RE)*, pp. 176–185, IEEE, 2016.

[2] J. Wei, A.-L. Courbis, T. Lambolais, B. Xu, P. L. Bernard, and G. Dray, "Zero-shot bilingual app reviews mining with large language models," Nov. 2023.

[3] E. Knauss, S. H. Houmb, S. Islam, J. Jürjens, and K. Schneider, "Secreq," Feb. 2021.

[4] A. Rezaei Nasab, M. Dashti, M. Shahin, M. Zahedi, H. Khalajzadeh, C. Arora, and P. Liang, "Fairness concerns in app reviews: A study on ai- based mobile apps," June 2024.

[5] S. R. Kasa and V. Rajan, "Avoiding inferior clusterings with misspecified gaussian mixture models," *Scientific Reports*, vol. 13, no. 1, p. 19164, 2023.